ChitLite - Complete Project & Interview Master Guide

This document is a comprehensive study guide designed to prepare you for technical and system design interviews based on the ChitLite project. It consolidates the project overview, system architecture, core business logic pseudocode, and common interview questions.

---

1. Project Overview & Core Features

What is ChitLite?

ChitLite is a digital management system for **Chit Funds** (rotating savings and credit associations). It replaces manual paper ledger systems with a real-time, responsive web portal.

User Roles & Key Flows:

1. **Admin/Owner**:
   - Creates chit groups (e.g., ₹300,000 value, 30 months, 30 members).
   - Enrolls members (individually or via bulk CSV upload).
   - Runs the **monthly bidding auction** (determines who gets the pool and calculates dividends/dues).
   - Manages the collection grid (dues matrix) and marks cash/bank payments manually.
   - Sends automated SMS (Twilio) and Email (Brevo) payment alerts.
2. **Client/Subscriber**:
   - Logs in using a secure Client ID and phone number.
   - Views their active groups, monthly dues, and winner status.
   - Pays outstanding bills online via **Razorpay Integration** (single payment or bulk checkout).
   - Submits support inquiries to the owner.

1.5 Core Web Concepts & Tech Stack Definitions

Before diving into the system design, it is essential to understand the core terminology and technology stack choices.

A. Authentication vs. Authorization vs. Auth Token

- **Authentication (AuthN) - "Who are you?"**:

- The process of verifying a user's identity. In ChitLite, this occurs when an admin enters their username and password, or when a client enters their Client ID and phone number.

- **Authorization (AuthZ) - "What are you allowed to do?"**:
  - The process of verifying permissions. Once authenticated, the system checks if you are an **Admin** (allowed to create groups, run auctions, and view log sheets) or a **Client** (only allowed to view your personal dues and pay bills).
- **Auth Token (Authentication Token)**:
  - A secure, signed string issued by the server upon successful authentication. Instead of sending credentials with every HTTP request, the client attaches this token (JWT) to the headers. The server verifies the token to identify and authorize the request.

B. Stateful vs. Stateless Authentication

- **Stateful Authentication (Sessions)**:
  - The server generates a session ID, stores it in memory or a database (e.g. Redis), and sends it to the client via cookies. The server must search the database on every request to validate the session.
- **Stateless Authentication (JWT - Used by us)**:
  - The server signs a payload containing user details (ID, role) and sends it to the client. The server does not store the token. When the client sends the token back, the server verifies the signature mathematically using its secret key. This is highly scalable because the server performs no database lookups for session validation.

C. Stack Comparison: What did we use and why?

In modern web development, teams often use predefined stacks:

- **MERN**: **M**ongoDB, **E**xpress, **R**eact, **N**ode.js.
- **MEAN**: **M**ongoDB, **E**xpress, **A**ngular, **N**ode.js.
- **Vite**: A modern frontend build tool (bundler) used to compile React/Vue single-page apps.

1. Why we chose PostgreSQL over MongoDB (SQL vs. NoSQL)

ChitLite manages financial transactions, ledger balances, and payment records.

- **PostgreSQL (SQL - Used by us)**: Relational database with strict **ACID compliance** (Atomicity, Consistency, Isolation, Durability). It enforces strict schemas, foreign key constraints (preventing deleting a member who has unpaid bills), and relational queries, which are vital for accounting integrity.
- **MongoDB (NoSQL - Used in MERN/MEAN)**: Document-oriented database. While flexible, it does not naturally enforce relational constraints or transactional boundaries across separate tables, making it less suitable for ledgers and financial applications.

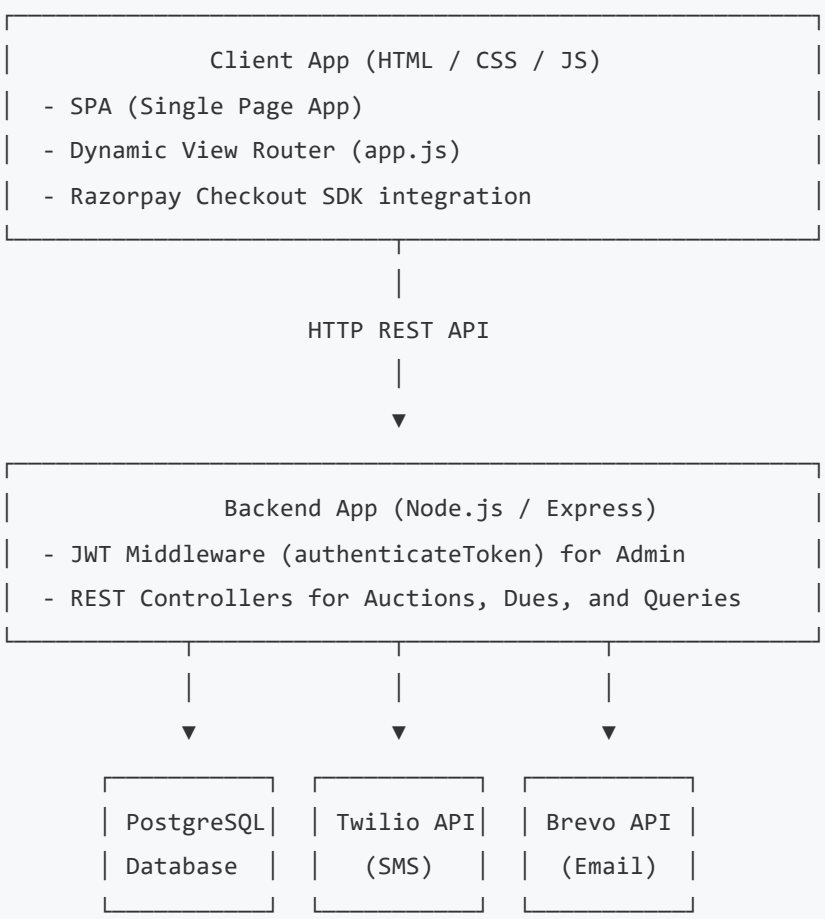
2. Why we chose Vanilla HTML5/CSS3/JS over React/Angular (MERN/MEAN)

Instead of building a heavy React/Angular SPA compiled with Vite, we built a **Vanilla JS SPA**:

- **Zero Build Compile Step**: React/Angular requires a bundler like **Vite** or Webpack to compile JSX/TSX code into static files before deploying. Vanilla JS runs natively in all browsers without compilation.
- **Render Deployment Speed**: Free cloud tiers on Render have strict memory limits. Compiling a React app on Render frequently runs out of memory or takes minutes. A Vanilla JS app deploys instantly because there is nothing to build.
- **Capacitor Mobile Friendly**: Capacitor wraps standard web assets (`public` folder) directly into native WebView folders. Wrapping a compiled React/Vite app requires setting up configuration files to prevent path errors (`dist/index.html` referencing absolute bundle scripts). Vanilla HTML/JS works natively with no additional paths.

---

2. Simplified System Design



Database Tables Schema:

- `groups` : Configurations (value, duration, current month, commission rate).
- `members` : Holds client details linked to specific groups. Has a unique `client_id` .
- `auctions` : Monthly bidding details (winner, discount bid, dividend distributed).
- `payments` : Individual monthly bills for each member (`paid` / `unpaid`).
- `notifications` : History of sent and failed alerts.
- `queries` : Customer support tickets.

---

3. Core Mathematical Formulas

Let V be the total group value, N be the number of members, $C%$ be the owner commission rate, and D be the winning bid discount.

1. Owner Commission Value:

$$Commission = V \times \frac{C}{100}$$

2. Dividend Pool:

The dividend pool is the winner's discount minus the owner's commission: $$DividendPool = D - Commission$$

3. Dividend Distributed Per Member:

$$DividendPerMember = \frac{D - Commission}{N}$$

4. Monthly Installment Due (per member):

$$MonthlyDue = \frac{V}{N} - DividendPerMember$$

---

4. Key Logic & Pseudocode

Here is the exact algorithmic logic for the most complex processes in the application.

A. Monthly Auction and Dividend Calculation

When an admin submits a winning bid discount for a month, the system must determine the winner, compute dividends, and generate payment bills for all members.

```
FUNCTION processMonthlyAuction(groupId, winnerMemberId, bidDiscount) {
    // 1. Fetch group configurations
    group = DB.query("SELECT * FROM groups WHERE id = groupId")
    N = group.duration
    V = group.value
    C = group.commission_rate
    M = group.current_month

    // 2. Perform financial calculations
    commissionVal = V * (C / 100)
    amountWonVal = V - bidDiscount
    dividendPool = bidDiscount - commissionVal
    dividendPerMember = dividendPool / N
    installmentAmount = (V / N) - dividendPerMember

    // 3. Start Database Transaction
    BEGIN TRANSACTION
    TRY
        // A. Insert Auction Log
        DB.query("INSERT INTO auctions (group_id, winner_member_id, month_number, bid_discount, di

        // B. Generate monthly bills for all members
        members = DB.query("SELECT * FROM members WHERE group_id = groupId")
        FOR EACH member IN members {
            DB.query("INSERT INTO payments (group_id, member_id, month_number, amount_due, status)
        }

        COMMIT TRANSACTION
        RETURN SUCCESS
    CATCH ERROR
        ROLLBACK TRANSACTION
        RETURN ERROR
}
```

B. Winner Payout and Backward Dues Deduction

When a member wins, they shouldn't receive the full gross amount won if they owe money elsewhere. The system automatically calculates outstanding dues across **all** groups and deducts them.

Additionally, past winners pay a small extra premium (`winnerExtraAmount`) to increase the pool for later winners.

```
FUNCTION calculateWinnerNetPayout(groupId, winnerMemberId, amountWonVal, commissionVal) {
    // 1. Add extra premiums from past winners
    pastWinners = DB.query("SELECT * FROM auctions WHERE group_id = groupId")
    pastWinnersCount = pastWinners.length
    winnerExtraAmount = 500 // Flat premium fee
    premiumIncrease = pastWinnersCount * winnerExtraAmount

    // 2. Calculate Winner's Gross Payout
    // Note: The winner still owes their own monthly due for this month
    winnerDue = DB.query("SELECT amount_due FROM payments WHERE group_id = groupId AND member_id =
    grossPayout = amountWonVal - winnerDue - commissionVal + premiumIncrease

    // 3. Find and deduct outstanding dues across ALL groups
    allUnpaidDues = DB.query("SELECT * FROM payments WHERE member_id = winnerMemberId AND status =

    totalDeductions = 0
    remainingPayout = grossPayout

    BEGIN TRANSACTION
    TRY
        FOR EACH due IN allUnpaidDues {
            IF remainingPayout >= due.amount_due {
                remainingPayout -= due.amount_due
                totalDeductions += due.amount_due
                // Mark as fully paid
                DB.query("UPDATE payments SET status = 'paid', remarks = 'Cleared via winner payou
            } ELSE IF remainingPayout > 0 {
                // Deduct remaining balance and mark as partially paid
                totalDeductions += remainingPayout
                DB.query("UPDATE payments SET amount_due = (due.amount_due - remainingPayout), rem
                remainingPayout = 0
            }
        }
        COMMIT TRANSACTION
    CATCH ERROR
        ROLLBACK TRANSACTION
        RETURN ERROR

    netReceivable = remainingPayout
    RETURN { grossPayout, totalDeductions, netReceivable }
}
```

C. Client Online Payment Checkout (Razorpay)

Prevents double-spending and confirms digital payments cryptographically.

```
// Step 1: Create Order (Backend)
FUNCTION createPaymentOrder(memberId, dueId) {
    dueRecord = DB.query("SELECT amount_due FROM payments WHERE id = dueId AND status = 'unpaid'")
    IF NOT dueRecord RETURN ERROR "Already Paid"

    // Create order with Razorpay Gateway
    options = {
        amount: dueRecord.amount_due * 100, // in paisa
        currency: "INR",
        receipt: dueId
    }
    razorpayOrder = RazorpaySDK.orders.create(options)
    RETURN razorpayOrder.id
}

// Step 2: Verify and Clear Dues (Backend)
FUNCTION verifySignatureAndConfirm(razorpayOrderId, razorpayPaymentId, razorpaySignature, dueId) {
    // Generate cryptographic HMAC-SHA256 signature
    generatedSignature = hmac_sha256(razorpayOrderId + "|" + razorpayPaymentId, RAZORPAY_SECRET)

    IF generatedSignature === razorpaySignature {
        // Safe Update
        DB.query("UPDATE payments SET status = 'paid', transaction_id = razorpayPaymentId, paid_at
        RETURN SUCCESS
    } ELSE {
        RETURN ERROR "Fraudulent Transaction"
    }
}
```

5. Top 10 Interview Questions & Answers

Q1: Explain how you bypassed Render's outbound SMTP block to deliver emails.

Answer: Render's free-tier containers block all outbound traffic on standard SMTP ports (25, 465, and 587) to prevent spam. Connecting to traditional mail servers using standard Nodemailer would result in a `Connection Timeout`. I bypassed this network block by switching to **Brevo's transactional HTTP API** over **Port 443 (HTTPS)**. Since port 443 is used for standard secure web traffic, it is fully open on Render. I rewrote the mail utility using Node's native `fetch` to POST JSON payloads directly to `https://api.brevo.com/v3/smtp/email` using an API Key header.

Q2: How did you ensure database data integrity (ACID) during the monthly bidding auction?

Answer: A monthly auction requires multiple database inserts and updates: inserting an auction entry, generating individual monthly dues for all members, and updating the group's current month counter. If the server crashes midway, some members might get bills while others do not. I ensured **Atomicity (All-or-Nothing)** by wrapping these queries in a PostgreSQL database **Transaction** (`BEGIN` , `COMMIT` , `ROLLBACK`). If any individual query fails, or a server error occurs, the entire block is rolled back to leave the database in its original clean state.

Q3: What is "Backward Dues Deduction" in your system and how is it implemented?

Answer: Backward Dues Deduction is a risk mitigation feature. When a subscriber wins an auction, they are eligible for a cash payout. However, they might have outstanding unpaid bills in this group or other chit groups they belong to. Before dispersing the winner's payout, the backend queries the database for all unpaid installments across all groups for that member. It automatically deducts these unpaid amounts from their gross payout, updates the status of those bills to `paid` with a remark, and calculates the remaining **Net Payout** to disperse to the client.

Q4: How does the client login work without passwords?

Answer: Chit funds are localized saving circles where owners register members offline. To keep login frictionless, clients do not need to register a password. Instead, they login using their **5-digit Client ID** (generated on enrollment) and their **registered phone number**. The backend verifies that a member exists matching both credentials in the database, and issues a signed JSON Web Token (JWT) containing the member's ID. The client stores this JWT in `localStorage` to authorize subsequent API requests.

Q5: How do you prevent double-payment or double-submission bugs in the UI?

Answer: If a user double-clicks a "Submit Bid" or "Pay Dues" button, it can trigger duplicate HTTP requests, resulting in duplicate groups, multiple auction records, or double credit card charges. I resolved this by implementing **Double-Click Protection** in the frontend:

1. When a form or button is clicked, a Javascript handler immediately adds a `disabled` attribute to the button and injects an active loading spinner.
2. The button remains disabled until the API request finishes (resolved or rejected).
3. If successful, the view changes; if it fails, the button is re-enabled and the spinner is removed, showing the error toast.

Q6: How does the app dynamically transition from a Web Portal to a Native Mobile App?

Answer: We wrapped our vanilla web frontend with **Capacitor**. When the app is bundled natively (using Capacitor's Android engine), the assets are served from the local file system (`file://` or custom local schemas). If it made relative API calls like `/api/groups`, the app would try to load `file:///api/groups` and crash. I made the API base path dynamic in `app.js`:

```
const API_BASE = (window.location.origin.startsWith('file://') || window.location.hostname === 'lc
  ? 'https://chitlite-portal.onrender.com/api'
  : '/api';
```

This detects if the application is running from a local file directory (mobile webview) and forces it to target the hosted Render server, while retaining relative path configurations when running on Render's web domain.

Q7: Explain the difference between connection pooling and single client connections in pg.

Answer: Creating a new TCP connection to PostgreSQL for every single HTTP request is extremely expensive and can cause database crashes due to connection exhaustion. In `db.js`, I configured a **Connection Pool** (`pg.Pool`). The pool maintains a reusable cache of active client connections. When a request comes in, it checks out an idle client, executes the query, and releases it back to the pool immediately. This enables high concurrency and optimal resource usage under load.

Q8: What security measures did you implement to protect the REST endpoints?

Answer:

1. **Route Authentication**: Admin endpoints require a valid JWT token in the `Authorization` header (`Bearer <token>`). A JWT middleware verifies the signature using a server-side secret key (`JWT_SECRET`).
2. **SQL Injection Prevention**: All queries use parameterized inputs (e.g. `$1, $2` placeholders) instead of string concatenation, preventing malicious SQL code execution.
3. **CORS Restrictions**: Express utilizes the `cors` middleware, allowing you to whitelist only authorized client origins from accessing backend APIs.

Q9: Tell me about a challenging bug you faced during the project and how you solved it.

Answer: *Example Answer:* During the email notification testing, we encountered an `ENETUNREACH` error on Render. The server couldn't resolve the SMTP server address. I investigated and found that Node's default DNS lookup prefers IPv6 addresses, but Render's internal containers do not have outgoing IPv6 network access configured on the free tier. I resolved this by forcing Nodemailer connections to prefer IPv4 first in DNS resolution (by setting `family: 4` inside the transporter lookup configuration). Later, when port blocking still interrupted SMTP traffic, I completely transitioned the mail system to Brevo's HTTP API over Port 443, which bypasses firewalls completely.

Q10: How did you implement bulk uploads of subscribers?

Answer: Instead of typing out dozens of members manually, administrators can download a template CSV file (`sample_members.csv`). The admin fills out the columns (Name, Phone, Email) and uploads the file. The backend parses the file line-by-line, runs validation (checks for valid email formats and phone numbers), generates unique 5-digit Client IDs for each member, and executes a batch insert into the database.

---

6. Core Production Source Code

Here are the actual code implementations directly from the ChitLite source codebase:

A. Database Connection Pool Setup (`db.js`)

Handles pooled connections with support for Supabase SSL requirements in production and localhost configurations in development.

```
const { Pool } = require('pg');

const pool = process.env.DATABASE_URL
  ? new Pool({
      connectionString: process.env.DATABASE_URL,
      ssl: { rejectUnauthorized: false }
    })
  : new Pool({
      host: process.env.DB_HOST || 'localhost',
      port: process.env.DB_PORT || 5432,
      database: process.env.DB_NAME || 'chitfund_db',
      user: process.env.DB_USER || 'postgres',
      password: process.env.DB_PASSWORD || '1234'
    });

// Shared query helper
async function query(text, params) {
  const start = Date.now();
  const res = await pool.query(text, params);
  const duration = Date.now() - start;
  if (process.env.NODE_ENV === 'development') {
    console.log('[Executing Query]:', { text, duration: `${duration}ms`, rows: res.rowCount });
  }
  return res;
}
```

B. JWT Authentication Middleware (`server.js`)

Intercepts and validates administrative client requests requesting access to protected endpoints.

```
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Access token required. Please login.' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, decodedUser) => {
    if (err) {
      return res.status(403).json({ error: 'Token expired or invalid.' });
    }
    req.user = decodedUser;
    next();
  });
}
```

C. Brevo Email & Twilio SMS Dispatchers (`server.js`)

Demonstrates how Twilio (via SDK) and Brevo (via custom HTTP REST POST) are implemented to deliver real-time notifications.

```
const twilio = require('twilio');
const twilioClient = process.env.TWILIO_ACCOUNT_SID
  ? twilio(process.env.TWILIO_ACCOUNT_SID, process.env.TWILIO_AUTH_TOKEN)
  : null;

async function dispatchNotification(groupId, member, type, message, isTest = false) {
  const notifId = crypto.randomUUID();
  let status = 'sent';
  let errorMsg = '';

  if (type === 'sms') {
    const recipientPhone = formatPhoneNumber(member.phone);
    if (twilioClient && !isTest) {
      try {
        await twilioClient.messages.create({
          body: message,
          to: recipientPhone,
          from: process.env.TWILIO_PHONE_NUMBER
        });
        console.log(`[Twilio SMS Sent to ${recipientPhone}]`);
      } catch (err) {
        status = 'failed';
        errorMsg = err.message;
      }
    }
  } else if (type === 'email') {
    const recipientEmail = member.email ? member.email.trim() : null;
    if (recipientEmail && !isTest) {
      try {
        if (process.env.BREVO_API_KEY) {
          // Bypasses Render outbound SMTP blocks using Port 443 HTTPS POST
          const brevoRes = await fetch('https://api.brevo.com/v3/smtp/email', {
            method: 'POST',
            headers: {
              'accept': 'application/json',
              'api-key': process.env.BREVO_API_KEY,
              'content-type': 'application/json'
            },
            body: JSON.stringify({
              sender: { name: 'ChitLite Portal', email: process.env.EMAIL_USER },
              to: [{ email: recipientEmail }],
              subject: 'Chit Fund Alert Details',
              textContent: message
```

```
            })
          });

          if (!brevoRes.ok) {
            const errData = await brevoRes.json().catch(() => ({}));
            throw new Error(`Brevo API status ${brevoRes.status}: ${JSON.stringify(errData)}`);
          }
        }
      } catch (err) {
        status = 'failed';
        errorMsg = err.message;
      }
    }
  }

  // Insert notification log to database
  await db.query(
    `INSERT INTO notifications (id, group_id, member_id, member_name, type, recipient, message, st
     VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)`,
    [notifId, groupId, member.id, member.name, type, recipient, message, status, errorMsg]
  );
}
```

D. Razorpay Checkout Order & Signature Verification (`server.js`)

Demonstrates how Razorpay handles payments secure checkout transactions and verifies signatures locally.

```
const Razorpay = require('razorpay');
const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET
});

// 1. Create Razorpay transaction order
app.post('/api/payments/create-order', async (req, res) => {
  const { paymentId } = req.body;
  try {
    const payRes = await db.query('SELECT * FROM payments WHERE id = $1', [paymentId]);
    const payment = payRes.rows[0];

    const amountInPaise = Math.round(Number(payment.amount_paid) * 100);
    const orderOptions = {
      amount: amountInPaise,
      currency: 'INR',
      receipt: payment.id
    };

    const order = await razorpay.orders.create(orderOptions);
    res.json({
      success: true,
      keyId: process.env.RAZORPAY_KEY_ID,
      orderId: order.id,
      amount: order.amount,
      paymentId: payment.id
    });
  } catch (error) {
    res.status(500).json({ error: 'Failed to initiate gateway transaction' });
  }
});

// 2. Cryptographically verify signature and mark paid
app.post('/api/payments/verify-signature', async (req, res) => {
  const { paymentId, razorpay_order_id, razorpay_payment_id, razorpay_signature } = req.body;

  try {
    const secret = process.env.RAZORPAY_KEY_SECRET;
    const body = razorpay_order_id + '|' + razorpay_payment_id;

    const expectedSignature = crypto
      .createHmac('sha256', secret)
```

```
      .update(body.toString())
      .digest('hex');

    if (expectedSignature !== razorpay_signature) {
      return res.status(400).json({ error: 'Payment signature verification failed.' });
    }

    // Safely update billing status in database
    await db.query(
      `UPDATE payments
       SET status = 'paid', payment_method = 'gateway_online', notes = $1, paid_at = NOW()
       WHERE id = $2`,
      [`Razorpay Pay ID: ${razorpay_payment_id}`, paymentId]
    );

    res.json({ success: true });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```